



Creación de un chatbot con IA capaz de buscar información en fuentes externas

Grado en Ingeniería Informática

Trabajo Fin de Grado

Autor:

Mario Martínez Turpín

Tutor/es:

Jose Antonio Miñarro Gimenez

Catalina Martínez Costa



**Facultad
Informática
Universidad
Murcia**

21 de Mayo de 2026

Creación de un chatbot con IA capaz de buscar información en fuentes externas

MMT Chat

Autor

Mario Martínez Turpín

Tutor/es

Jose Antonio Miñarro Gimenez

Departamento de Informática y Sistemas

Catalina Martínez Costa

 **Facultad de
Informática**

Grado en Ingeniería Informática

CIIC

UNIVERSIDAD DE
MURCIA



Murcia, 21 de Mayo de 2026

Agradecimientos

Me gustaría expresar mi agradecimiento a mis tutores, Jose Antonio Miñarro Gime-
nez y Catalina Martínez Costa, por su guía a lo largo de todo el proceso. Su ayuda ha
sido fundamental para navegar los procedimientos formales y darle forma al trabajo.

Asimismo, quiero agradecer a mi familia por el apoyo incondicional y la confianza
que han depositado en mí durante todo este tiempo. Sin ellos, este proyecto no habría
sido posible.

También quiero destacar a mi pareja, Natalia, por su amor, paciencia y comprensión,
que tanto me han ayudado a seguir adelante durante estos meses.

Finalmente, dedico este trabajo a todas las personas que han contribuido a que
esta experiencia sea enriquecedora y exitosa. Su ayuda y confianza me han permitido
alcanzar mis objetivos y convertir esta tesis en una realidad.

Declaración firmada sobre originalidad del trabajo

D./Dña. **Mario Martínez Turpín**, con DNI **49183039T**, estudiante de la titulación de **Grado en Ingeniería Informática** de la Universidad de Murcia y autor del TF titulado “**Creación de un chatbot con IA capaz de buscar información en fuentes externas**”.

De acuerdo con el Reglamento por el que se regulan los Trabajos Fin de Grado y de Fin de Máster en la Universidad de Murcia (aprobado C. de Gob. 30-04-2015, modificado 22-04-2016 y 28-09-2018), así como la normativa interna para la oferta, asignación, elaboración y defensa de los Trabajos Fin de Grado y Fin de Máster de las titulaciones impartidas en la Facultad de Informática de la Universidad de Murcia (aprobada en Junta de Facultad 27-11-2015)

DECLARO:

Que el Trabajo Fin de Grado presentado para su evaluación es original y de elaboración personal. Todas las fuentes utilizadas han sido debidamente citadas. Así mismo, declara que no incumple ningún contrato de confidencialidad, ni viola ningún derecho de propiedad intelectual e industrial

Murcia, a 21 de Mayo de 2026



Fdo.: Mario Martínez Turpín
Autor del TF

Resumen

En la era digital actual, el acceso eficiente y preciso a grandes volúmenes de información estructurada y no estructurada representa uno de los mayores desafíos en el ámbito de las Ciencias de la Computación. Históricamente, la Web Semántica y los Grafos de Conocimiento (*Knowledge Graphs*) han proporcionado un marco tecnológico robusto para modelar información de manera determinista mediante tripletas (Sujeto, Predicado, Objeto) basadas en el estándar RDF (Resource Description Framework). El acceso a esta información altamente estructurada y veraz se realiza tradicionalmente a través de SPARQL, un lenguaje de consulta potente pero con una curva de aprendizaje pronunciada que supone una barrera técnica insalvable para la mayoría de los usuarios no expertos.

De forma paralela, el reciente auge de los Modelos de Lenguaje de Gran Escala (LLMs) ha revolucionado la interacción humano-máquina. Estos modelos permiten establecer interfaces conversacionales fluidas en lenguaje natural, facilitando enormemente el acceso a la información. Sin embargo, su naturaleza estocástica los hace vulnerables a problemas inherentes como las denominadas "alucinaciones" (la generación de respuestas que resultan gramaticalmente correctas y plausibles, pero que son total o parcialmente falsas) y presentan serias dificultades para recuperar datos precisos y actualizados de dominios de conocimiento altamente específicos.

El presente Trabajo de Fin de Grado (TFG) aborda la convergencia de ambas tecnologías con el objetivo de aunar la flexibilidad y capacidad de comprensión del lenguaje natural propia de los LLMs con la precisión factual inmutable de un Grafo de Conocimiento. Para lograr este ambicioso objetivo, se ha diseñado, implementado y evaluado un agente conversacional inteligente (chatbot) especializado en el dominio cinematográfico. A diferencia de un chatbot tradicional que responde basándose puramente en los patrones aprendidos en su entrenamiento, este sistema actúa como una interfaz transparente que traduce la intención del usuario a consultas SPARQL válidas, ejecutándolas contra una base de datos local y devolviendo la respuesta exacta.

La arquitectura del sistema propuesto se fundamenta en una evolución del paradigma de Generación Aumentada por Recuperación (RAG), elevándolo hacia un sistema de Inteligencia Artificial Agéntica utilizando el *framework* LangChain. La solución tecnológica se ha estructurado en tres capas principales. En la capa de presentación, se ha desarrollado una interfaz gráfica de usuario amigable mediante Streamlit, que no solo facilita la interacción conversacional, sino que también expone de manera transparente el razonamiento interno del agente ("Chain-of-Thought") y las consultas generadas en tiempo real. En la capa lógica, el núcleo cognitivo del sistema, impulsado por el

modelo Llama3 ejecutado localmente mediante Ollama, se encarga de la extracción de entidades, su desambiguación contra el vocabulario de Wikidata y la generación dinámica de código SPARQL. Finalmente, la capa de datos descansa sobre un Grafo de Conocimiento local gestionado mediante la librería RDFLib en Python.

Un hito técnico fundamental de este trabajo ha sido la construcción de este repositorio de datos semántico. Depender directamente de *endpoints* públicos como el de Wikidata durante la ejecución en tiempo real introducía niveles inaceptables de latencia y riesgo de bloqueos por límites de peticiones (APIs *rate limits*). Por ello, se implementó un proceso de extracción *offline* estructurado para aislar un subgrafo densamente conectado del dominio cinematográfico (abarcando miles de películas, directores, actores, géneros y fechas de estreno), el cual fue serializado en formato *Turtle* (.ttl). Esto proporciona al agente un entorno de ejecución local, excepcionalmente rápido y completamente determinista.

El aspecto más innovador de la implementación radica en el mecanismo autónomo de autocorrección del agente, denominado "bucle de reflexión" (*Reflexion Loop*). La traducción de lenguaje natural a sintaxis SPARQL estricta es propensa a errores. Para mitigarlo, el agente cuenta con un diccionario de propiedades permitidas altamente restrictivo (por ejemplo, `wdt:P57` exclusivamente para referenciar a un "director") que acota drásticamente su espacio de búsqueda. Si el modelo genera una consulta con errores de sintaxis o que no compila, la capa de datos captura la excepción (*traceback*) de RDFLib y, en lugar de fallar de manera abrupta, realimenta al LLM con este error de compilación inyectado en un *prompt* de reflexión. De este modo, el agente analiza lógicamente su propia equivocación y reescribe la consulta desde cero, logrando un notable incremento en la robustez, resiliencia y autonomía del sistema.

Para validar el sistema propuesto de forma objetiva, rigurosa y cuantitativa, se desarrolló ad-hoc un marco de evaluación automatizado. Se confeccionó un conjunto de pruebas (*dataset*) compuesto por 61 consultas complejas en lenguaje natural que abarcaban diversas relaciones semánticas de dificultad incremental, y se llevó a cabo un estudio de rendimiento comparativo entre distintos modelos de lenguaje de gran escala. El modelo seleccionado para la versión final del proyecto, Llama3, demostró un rendimiento excepcional: alcanzó una tasa de precisión del 90.16% en la fase crítica de extracción y desambiguación de entidades, e idéntico 90.16% de éxito global en la generación de código SPARQL válido y ejecutable que retornó la respuesta correcta a la pregunta planteada.

Además de las tasas de éxito en precisión, se midió exhaustivamente el tiempo de ejecución por pregunta, un factor sumamente crítico para asegurar una buena experiencia de usuario en cualquier asistente conversacional interactivo. Llama3 demostró un rendimiento óptimo en este aspecto, promediando un tiempo de ejecución de respuesta de tan solo 3.66 segundos por pregunta, erigiéndose de manera destacada como el modelo más equilibrado y rápido de entre todos los evaluados en este trabajo. Un análisis pormenorizado de los escasos fallos residuales reveló que estos se debían, en su inmensa mayoría, a ambigüedades idiomáticas extremas por parte del usuario o a

cuellos de botella aislados en las llamadas de desambiguación de entidades, y no a fallos estructurales o lógicos en la formulación de consultas deductivas por parte del LLM.

En conclusión, los resultados empíricos obtenidos a lo largo de las pruebas confirman sin lugar a dudas que la integración de agentes de LangChain fuertemente tipados con Grafos de Conocimiento locales constituye una solución tecnológica sumamente viable y efectiva para la creación de sistemas de pregunta-respuesta (QA) totalmente transparentes y estructuralmente libres de alucinaciones. El sistema diseñado ha logrado el hito de ocultar la enorme complejidad inherente al modelo de datos de la Web Semántica de cara al usuario final, manteniendo no obstante intacta la rigurosa precisión y veracidad de sus datos subyacentes. El presente trabajo sienta, por tanto, unas bases sólidas y prometedoras para futuras líneas de investigación y desarrollo, tales como la mejora de los algoritmos basados en similitud de vectores (*embeddings*) para lograr una desambiguación semántica libre de fallos, o la ampliación escalonada de la ontología base para llegar a soportar razonamientos deductivos mucho más complejos de múltiples saltos lógicos (*multi-hop reasoning*) que permitan extraer conclusiones de datos aparentemente inconexos.

Palabras clave: Web Semántica, Grafos de Conocimiento, SPARQL, Modelos de Lenguaje (LLMs), Llama3, Generación Aumentada por Recuperación (RAG), LangChain, Inteligencia Artificial Agéntica, Streamlit, Mitigación de Alucinaciones.

Extended Abstract

This Final Degree Project presents the comprehensive design, implementation, and rigorous evaluation of an intelligent conversational agent specialized in the cinematographic domain. The primary objective is to bridge the significant technical gap between the natural language comprehension capabilities of modern Large Language Models (LLMs) and the factual, deterministic precision inherent to Semantic Web Knowledge Graphs. By synergizing these two distinct technological paradigms, the project directly addresses one of the most critical and widely discussed challenges in modern generative Artificial Intelligence: the mitigation and elimination of "hallucinations"—the generation of plausible yet entirely fabricated information.

1. Introduction and Motivation

In recent years, LLMs have revolutionized the way users and developers interact with information systems, enabling highly fluent conversational interfaces. However, their stochastic nature makes them inherently unreliable for querying specific, factual, or specialized data, as they are prone to generating incorrect answers with high confidence. On the other hand, Knowledge Graphs and the Semantic Web provide impeccably structured, verifiable data environments accessible via precise query languages like SPARQL. The steep learning curve and strict syntax of SPARQL prevent non-expert users from accessing this immense wealth of information. This project proposes a robust hybrid solution: an autonomous conversational agent that acts as a transparent natural language interface for a local Knowledge Graph, intelligently translating user queries into SPARQL, executing them locally, and returning the exact data in a human-readable format.

2. State of the Art

The theoretical foundation of this work rests firmly on three technological pillars: the Semantic Web, Large Language Models, and Retrieval-Augmented Generation (RAG) paradigms. The Semantic Web, formalized and standardized by the W3C, relies heavily on the Resource Description Framework (RDF) to elegantly model complex information as subject-predicate-object triples. Wikidata, a massive, multilingual, and collaborative knowledge base, serves as the primary and authoritative data source for this project.

To effectively overcome the static knowledge limitations and hallucination issues of standard LLMs, RAG architectures dynamically retrieve external data and inject it into the model’s contextual window. This project pushes the traditional RAG paradigm significantly further by implementing an “Agentic AI” workflow using the LangChain framework. Within this architecture, the LLM is not relegated to merely retrieving text; it actively reasons, plans, writes code, tests queries against the database, and refines its approach.

3. System Architecture and Methodology

Given the highly experimental and empirical nature of advanced prompt engineering, an agile, iterative, and test-driven methodology was adopted throughout the development lifecycle. The overall system architecture is modularly divided into three highly cohesive main layers:

- **Presentation Layer:** A highly interactive and responsive graphical user interface built entirely with Streamlit. It provides a seamless chat-like experience while simultaneously displaying the internal reasoning process (Chain-of-Thought) and the generated SPARQL code to the user, ensuring maximum transparency.
- **Logic and Agent Layer:** The cognitive core of the application, powered by LangChain and Llama3 running locally via Ollama. This layer handles the complex tasks of natural language understanding, entity extraction, Wikidata term disambiguation, and the core iterative SPARQL generation loop.
- **Data Layer:** A highly optimized, in-memory RDF local repository serialized in `.ttl` format, managed by the RDFLib Python library. It contains a densely populated subset of domain-specific cinematic data.

4. Local Knowledge Graph Construction

Relying directly on public SPARQL endpoints (such as the Wikidata Query Service) during real-time user interaction introduces unacceptable latency and severe risks of being blocked by strict API rate limits. Therefore, a custom, highly tailored data extraction pipeline was developed. The system programmatically queries Wikidata entirely offline to extract a densely connected, self-contained subgraph focusing specifically on movies, directors, actors, genres, and release dates. This data is then serialized into the *Turtle* format. This approach provides a lightning-fast, secure, and fully controlled environment for the agent to execute queries without relying on unstable external network dependencies.

5. The Reflexion Loop

Arguably the most innovative and impactful aspect of the system’s implementation is the agent’s autonomous, self-healing error-correction mechanism. Translating fluid natural language into strict, unforgiving SPARQL syntax is a heavily error-prone task for any LLM. To guide the model, the agent is provided with a strict, curated dictionary of allowed RDF properties (e.g., `wdt:P57` exclusively for “director”).

When the agent confidently generates a SPARQL query, the Logic Layer attempts to execute it locally against the Data Layer via RDFLib. If a syntax or execution error occurs, the process does not fail catastrophically. Instead, the runtime traceback is intercepted, parsed, and fed back to the LLM in a specialized “reflexion prompt”. This prompt instructs the model to critically analyze its own previous mistake, understand the syntax error, and autonomously rewrite the query. This iterative self-correction loop drastically increases the overall robustness, reliability, and success rate of the system.

6. Evaluation and Results

To quantitatively validate the system’s performance and viability, a comprehensive automated evaluation framework was built. A rigorous dataset of 61 complex natural language queries was carefully created, covering a wide array of semantic relationships and designed specifically to compare the performance of various large language models. The chosen baseline model for this project (Llama3) achieved an outstanding 90.16% accuracy rate in the crucial steps of entity extraction and disambiguation. Furthermore, it also achieved a 90.16% overall success rate in generating valid, executable SPARQL queries that returned the factually correct answer.

Within this framework, we also rigorously evaluated the execution latency per question. Llama3 demonstrated its ideal suitability for real-time conversational AI applications by achieving a remarkable average execution time of just 3.66 seconds per question, which was comfortably the lowest latency among all the sophisticated models tested.

An in-depth analysis of the remaining failure cases revealed that most issues stemmed from extreme idiomatic ambiguities in the user’s phrasing or minor, residual API rate limits encountered during the entity disambiguation phase, rather than fundamental logical reasoning failures by the LLM itself. The extraordinarily high success rate in autonomous query generation conclusively proves that heavily constraining the LLM’s vast search space through strict prompt engineering and predefined property dictionaries is a highly effective, production-ready strategy.

7. Conclusions

The empirical results confirm beyond doubt that integrating powerful LangChain agents with strongly-typed, local Knowledge Graphs is a highly viable and superior technical solution for building transparent, hallucination-free Question-Answering systems. The resulting agentic workflow successfully and completely hid the underlying complexity of the Semantic Web from the end-user, all while retaining and leveraging its absolute factual precision. Future work will strategically focus on improving the semantic similarity algorithms to ensure flawless entity disambiguation, and expanding the underlying ontology to comfortably support highly complex, multi-hop reasoning questions.

Keywords: Semantic Web, Knowledge Graphs, SPARQL, Large Language Models, Llama3, RAG, LangChain, Agentic AI, Hallucination Mitigation, Streamlit.

Índice general

1	Introducción	1
1.1	Motivación	1
1.2	Objetivos	2
1.3	Estructura de la Memoria	2
2	Estado del Arte	4
2.1	Web Semántica y Grafos de Conocimiento	4
2.1.1	Resource Description Framework (RDF)	4
2.1.2	SPARQL	4
2.1.3	Wikidata	4
2.2	Modelos de Lenguaje de Gran Escala (LLMs)	5
2.3	Generación Aumentada por Recuperación (RAG)	5
2.4	Agentes Autónomos y LangChain	6
3	Análisis de objetivos y metodología	7
3.1	Metodología de Desarrollo	7
3.2	Análisis de Requisitos	7
3.2.1	Requisitos Funcionales	7
3.2.2	Requisitos No Funcionales	8
3.3	Casos de Uso	9
4	Diseño y resolución del trabajo realizado	11
4.1	Arquitectura del Sistema	11
4.2	Diseño del Agente Autocorrector	12
4.3	Diseño del Grafo de Conocimiento Local	13
5	Desarrollo e Implementación	14
5.1	Entorno de Desarrollo y Tecnologías	14
5.2	Extracción de Datos y Grafo Local	14
5.3	El Agente Conversacional y el Control del LLM	14
5.4	Bucle de Reflexión y Autocorrección	15
5.5	Interfaz Gráfica	15
6	Evaluación y Resultados	17
6.1	Entorno de Pruebas	17
6.2	Resultados Obtenidos	17
6.3	Análisis y Discusión de los Resultados	18
6.4	Conclusiones de la Evaluación	19
7	Conclusiones y Trabajo Futuro	20
7.1	Consecución de Objetivos	20
7.2	Líneas de Trabajo Futuro	20

Bibliografía**22**

Índice de figuras

3.1	Diagrama de Secuencia Principal	9
3.2	Diagrama de Secuencia con Bucle de Autocorrección	10
4.1	[AÑADIR DIAGRAMA DE ARQUITECTURA AQUÍ]	11
4.2	[AÑADIR DIAGRAMA DE FLUJO DEL AGENTE AQUÍ]	12

Índice de tablas

6.1	Comparativa de rendimiento, precisión y latencia entre distintos LLMs locales evaluados.	17
-----	--	----

Índice de Códigos

5.1	Inyección de mapeo de propiedades en el prompt	15
5.2	Bucle de reflexión ante errores de sintaxis	15

1 Introducción

En la actualidad, el acceso a grandes volúmenes de información estructurada y no estructurada se ha convertido en un desafío fundamental en el ámbito de las Ciencias de la Computación. En el contexto de la Web Semántica, los Grafos de Conocimiento (*Knowledge Graphs*) ofrecen un marco robusto para representar información mediante tripletas (Sujeto, Predicado, Objeto), permitiendo realizar consultas complejas mediante el lenguaje estándar SPARQL. Sin embargo, la barrera técnica que supone dominar este lenguaje limita el acceso a esta información para usuarios no expertos.

Por otro lado, el auge de los Modelos de Lenguaje de Gran Escala (LLMs) ha transformado la forma en que interactuamos con los sistemas de información, permitiendo interfaces conversacionales en lenguaje natural. No obstante, los LLMs sufren de problemas inherentes como las alucinaciones (generación de información plausible pero falsa) y la dificultad para recuperar datos precisos y actualizados de dominios específicos.

Este Trabajo de Fin de Grado (TFG) aborda la convergencia de ambas tecnologías mediante el desarrollo de un agente conversacional (chatbot) especializado en el dominio cinematográfico. El sistema combina la fluidez y capacidad de comprensión del lenguaje natural de los LLMs con la precisión de un Grafo de Conocimiento propio basado en Wikidata.

Para lograrlo, se ha diseñado un agente capaz de interpretar la intención del usuario, mapear entidades a identificadores semánticos y generar autónomamente consultas SPARQL válidas, pudiendo llegar a corregirse a sí mismo en caso de error gracias a un bucle de reflexión.

1.1 Motivación

La motivación principal de este proyecto surge de la necesidad de facilitar la exploración de datos semánticos sin requerir conocimientos técnicos previos. Extraer información concreta de un grafo RDF con millones de nodos (como es el caso del dominio del cine en Wikidata) resulta una tarea ardua.

Si bien los LLMs pueden responder preguntas de cultura general sobre cine, su conocimiento es estático y propenso a errores. Integrar un LLM como interfaz a un Grafo de Conocimiento mediante una arquitectura basada en la generación de SPARQL no solo asegura la veracidad de la respuesta, sino que permite trazar el origen del dato y justificar la respuesta.

Además, este proyecto busca explorar la complejidad técnica que supone dominar y restringir la salida de un LLM. Lograr que un modelo de lenguaje actúe no como un mero generador de texto libre, sino como un agente lógico que respeta esquemas estrictos (ontologías) y corrige iterativamente su propio código SPARQL ante errores sintácticos, representa un reto de ingeniería de software y de *prompt engineering* altamente motivador.

1.2 Objetivos

El objetivo general de este proyecto es diseñar, desarrollar y evaluar un sistema de pregunta-respuesta (QA) conversacional basado en Grafos de Conocimiento capaz de traducir consultas en lenguaje natural a consultas SPARQL precisas y ejecutables sobre una base de datos propia.

Para alcanzar este fin, se definen los siguientes objetivos específicos:

- **Precisión en la Generación de SPARQL:** Desarrollar un agente autónomo que maximice la precisión al traducir lenguaje natural a SPARQL, siendo capaz de gestionar un bucle de autocorrección (reflexión) para enmendar errores sintácticos o semánticos generados en intentos previos.
- **Manejo Avanzado de Datos RDF:** Extraer, procesar y almacenar un conjunto significativo de datos provenientes de Wikidata en formato *Turtle* para conformar un Grafo de Conocimiento local optimizado, demostrando soltura en el manejo de tecnologías de la Web Semántica.
- **Control y Restricción del LLM:** Diseñar un marco robusto de *prompting* estructurado y mapeo de propiedades que permita "restringir" y guiar el razonamiento del modelo de lenguaje, minimizando las alucinaciones y forzándolo a utilizar únicamente las propiedades y entidades definidas en el contexto.
- **Evaluación Cuantitativa:** Implementar un marco de evaluación automática que permita medir objetivamente el rendimiento del sistema en términos de extracción de entidades correctas y generación de consultas SPARQL válidas frente a un conjunto de pruebas.

1.3 Estructura de la Memoria

El resto de esta memoria se estructura de la siguiente manera:

En el **Capítulo 2** se presenta el Estado del Arte, donde se exploran los fundamentos teóricos de la Web Semántica, los Modelos de Lenguaje y las arquitecturas de Generación Aumentada por Recuperación (RAG) e Inteligencia Artificial Agéntica.

El **Capítulo 3** detalla el Análisis del sistema, describiendo los requisitos técnicos y los casos de uso principales.

El **Capítulo 4** expone el Diseño de la solución, abordando la arquitectura del sistema, el diseño del Grafo de Conocimiento y el flujo de ejecución del agente iterativo.

El **Capítulo 5** describe la Implementación del proyecto, profundizando en las decisiones de programación, el entorno tecnológico utilizado (Python, LangChain, Streamlit) y el despliegue del sistema mediante Docker.

En el **Capítulo 6** se presenta la Evaluación del sistema, analizando las métricas obtenidas tras someter al agente a diversas baterías de pruebas automatizadas.

Finalmente, el **Capítulo 7** recoge las Conclusiones extraídas del desarrollo, evaluando el grado de cumplimiento de los objetivos y proponiendo líneas de trabajo futuro.

2 Estado del Arte

Para comprender la arquitectura y las decisiones de diseño adoptadas en este Trabajo de Fin de Grado, es necesario establecer una base teórica sobre las tecnologías subyacentes. Este capítulo revisa los conceptos clave en los que se apoya el sistema desarrollado: la Web Semántica, los Modelos de Lenguaje de Gran Escala (LLMs) y los paradigmas emergentes de Inteligencia Artificial como la Generación Aumentada por Recuperación (RAG) y los agentes autónomos.

2.1 Web Semántica y Grafos de Conocimiento

La Web Semántica, propuesta originalmente por Tim Berners-Lee, busca extender la web actual dotando a la información de un significado bien definido, permitiendo que ordenadores y personas trabajen en cooperación. El núcleo de esta visión son los Grafos de Conocimiento (*Knowledge Graphs*), estructuras de datos que representan entidades del mundo real y sus interrelaciones de forma explícita.

2.1.1 Resource Description Framework (RDF)

RDF es el estándar principal para modelar información en la Web Semántica [1]. La información se estructura en forma de tripletas que representan una relación entre dos entidades, por lo que el modelo de datos está estructurado en forma de grafo, donde cada nodo es una entidad y cada arista es una relación entre dos entidades: *Sujeto*, *Predicado* y *Objeto*. Estas tripletas pueden serializarse en distintos formatos, siendo *Turtle* (Terse RDF Triple Language) uno de los más legibles por humanos y el formato elegido en este proyecto para almacenar la base de conocimiento local sobre el dominio del cine.

2.1.2 SPARQL

SPARQL (*SPARQL Protocol and RDF Query Language*) es el lenguaje de consulta estándar para bases de datos RDF [2]. Permite realizar consultas complejas sobre grafos, filtrar resultados y explorar las relaciones semánticas entre entidades. Aunque es extremadamente potente y preciso, su curva de aprendizaje es muy pronunciada, lo que justifica la necesidad de construir interfaces en lenguaje natural para democratizar el acceso a los datos, como se propone en este TFG.

2.1.3 Wikidata

Wikidata es una base de conocimiento libre, colaborativa y multilingüe que actúa como almacenamiento central para los datos estructurados de sus proyectos hermanos, como Wikipedia [3]. Dado su enorme volumen de datos y su estructura semántica abierta y en constante evolución, Wikidata se ha consolidado como una de las fuentes

de conocimiento general más importantes del mundo, sirviendo como origen de los datos sobre la industria del cine extraídos para este proyecto.

2.2 Modelos de Lenguaje de Gran Escala (LLMs)

Los Modelos de Lenguaje de Gran Escala (LLMs), como la familia GPT de OpenAI o los modelos LLaMA de Meta, son redes neuronales masivas basadas en la arquitectura *Transformer* [4], entrenadas sobre cantidades ingentes de texto. Estos modelos han demostrado capacidades sin precedentes en la comprensión, síntesis y generación de lenguaje natural.

Sin embargo, en el contexto de la recuperación de información real y precisa, los LLMs presentan dos limitaciones críticas inherentes a su naturaleza estocástica:

1. **Conocimiento estático:** El modelo solo conoce la información disponible hasta el momento de su entrenamiento, requiriendo reentrenamientos costosos para actualizarse.
2. **Alucinaciones:** Tendencia a generar respuestas altamente plausibles a nivel sintáctico pero incorrectas cuando el modelo no dispone de la información requerida en sus pesos internos.

Para mitigar estos problemas, especialmente en aplicaciones de dominio específico, es estrictamente necesario acoplar el LLM con fuentes de conocimiento externas fidedignas.

2.3 Generación Aumentada por Recuperación (RAG)

La Generación Aumentada por Recuperación (*Retrieval-Augmented Generation*, RAG) es una arquitectura [5] que mejora sustancialmente la fiabilidad de las respuestas de un LLM al integrar un sistema de recuperación de información y al obligar al LLM a basar sus respuestas en la información específica proporcionada en el momento de la consulta. En un flujo RAG estándar:

1. El usuario realiza una pregunta en lenguaje natural.
 2. El sistema busca información relevante en una base de datos externa.
 3. La información recuperada se inyecta en el *prompt* (contexto) del LLM.
 4. El LLM formula la respuesta final basándose en el contexto documentado proporcionado, en lugar de depender únicamente de su memoria interna.
-

En este TFG, se aplica una variante más estructurada de RAG, a menudo referida como **Graph RAG** o **Text-to-SPARQL RAG**. En lugar de recuperar fragmentos de texto mediante similitud vectorial (que puede ser imprecisa), el modelo se encarga de generar el código lógico (SPARQL) necesario para interrogar un Grafo de Conocimiento de forma determinista, garantizando respuestas estructuradas y exactas.

2.4 Agentes Autónomos y LangChain

Un paso más allá del uso estático de LLMs es el paradigma de la Inteligencia Artificial Agéntica. Un agente de IA es un sistema que utiliza un LLM como su "motor de razonamiento" central para decidir dinámicamente qué acciones tomar, qué herramientas utilizar y en qué orden.

A diferencia de un flujo de código secuencial clásico, un agente puede observar el resultado de una acción y reaccionar. Por ejemplo, si el agente genera una consulta SPARQL que, al ejecutarse en el motor de base de datos, devuelve un error de sintaxis, el agente puede leer dicho error, reflexionar sobre el fallo, autocorregir el código y volver a intentar la consulta. Este bucle de retroalimentación dota al sistema de una robustez crítica frente a la variabilidad de las respuestas del modelo.

LangChain [6] es el *framework* de desarrollo de código abierto líder en el ecosistema para la creación de aplicaciones impulsadas por modelos de lenguaje. Proporciona las abstracciones necesarias para construir cadenas de procesamiento complejas, integrar herramientas externas, gestionar la memoria de las conversaciones y orquestar el flujo cognitivo de agentes como el implementado en este proyecto para interactuar con la base de conocimiento local.

3 Análisis de objetivos y metodología

Este capítulo detalla el análisis previo a la implementación del sistema, describiendo la metodología de trabajo adoptada y los requisitos funcionales y no funcionales que han guiado el desarrollo del chatbot basado en Grafos de Conocimiento.

3.1 Metodología de Desarrollo

Dada la naturaleza exploratoria de este Trabajo de Fin de Grado, centrado en la integración de Inteligencia Artificial Generativa y Web Semántica, la adopción de una metodología clásica en cascada (*Waterfall*) resultaba inapropiada. El comportamiento de los Modelos de Lenguaje de Gran Escala (LLMs) es no determinista y requiere de experimentación continua, especialmente en el ámbito de la ingeniería de *prompts* y la extracción de datos.

Por ello, se ha optado por un **enfoque ágil e iterativo basado en prototipos**. El desarrollo se ha dividido en ciclos cortos de experimentación:

1. **Exploración y Extracción de Datos:** Pruebas iniciales para consultar Wikidata y consolidar un Grafo de Conocimiento local funcional y acotado.
2. **Prototipado del Agente:** Desarrollo iterativo del *prompt* principal del LLM para traducir lenguaje natural a SPARQL.
3. **Implementación del Bucle de Reflexión:** Adición del mecanismo de autocorrección ante errores de sintaxis en el código generado.
4. **Evaluación y Refinamiento:** Pruebas de rendimiento frente al *dataset* de evaluación que derivaron en ajustes continuos en el flujo de ejecución.
5. **Implementación del Frontend:** Desarrollo de un interfaz de usuario para la interacción con el sistema.

Este enfoque ha permitido volver a pasos anteriores en caso de que fuera necesario, permitiendo un mayor desarrollo de las estrategias de *prompting* y una mayor integración de los componentes del sistema.

3.2 Análisis de Requisitos

Para asegurar que el sistema cumple con los objetivos planteados, se han especificado una serie de requisitos funcionales y no funcionales.

3.2.1 Requisitos Funcionales

Los requisitos funcionales (RF) describen las interacciones y el comportamiento esperado del sistema:

- **RF-01. Interfaz conversacional:** El sistema debe proporcionar una interfaz gráfica que permita al usuario realizar preguntas en lenguaje natural de forma intuitiva, similar a un chat convencional.
- **RF-02. Generación de la base de conocimiento local:** El sistema debe disponer de un módulo para extraer entidades de Wikidata.
- **RF-03. Extracción y desambiguación de entidades:** El sistema debe ser capaz de identificar la entidad principal en la consulta del usuario (ej. nombre de un actor o película) para obtener su identificador único (ej. `wd:Q25191`).
- **RF-04. Generación de SPARQL:** El agente LLM debe traducir la pregunta del usuario en una consulta SPARQL sintáctica y semánticamente válida, usando la entidad extraída y el diccionario de propiedades del sistema.
- **RF-05. Obtención de resultados:** El sistema debe ser capaz de obtener los resultados de la consulta SPARQL a partir de la base de datos local.
- **RF-06. Bucle de autocorrección:** Si la consulta SPARQL generada produce un error al ejecutarse en el motor RDF, el agente debe leer el mensaje de error y su propia consulta para intentar corregirla de forma autónoma.
- **RF-07. Ejecución y respuesta:** El sistema debe presentar la respuesta extraída al usuario en un formato legible.
- **RF-08. Mantenimiento del contexto:** El sistema debe ser capaz de resolver correferencias, para poder responder a preguntas como "¿Qué otras películas ha protagonizado?" sin necesidad de repetir el nombre del actor.
- **RF-09. Transparencia y Explicabilidad:** El sistema debe mostrar al usuario el razonamiento interno, la detección de entidades y el código SPARQL generado en tiempo real.

3.2.2 Requisitos No Funcionales

Los requisitos no funcionales (RNF) definen las restricciones y atributos de calidad del sistema:

- **RNF-01. Precisión y mitigación de alucinaciones:** El sistema debe priorizar la exactitud del código SPARQL generado, limitando estrictamente el alcance de conocimiento del LLM al esquema de la base de datos local para evitar alucinaciones.
 - **RNF-02. Privacidad y Ejecución Local:** El sistema debe garantizar la total privacidad de las consultas y la independencia de APIs externas de pago o en la nube para funcionar.
-

- **RNF-03. Tiempos de respuesta:** El sistema debe mantener tiempos de respuesta aceptables (generalmente inferiores a los 10 segundos), teniendo en cuenta el tiempo requerido para las llamadas a la API del LLM y la desambiguación.
- **RNF-04. Modularidad:** El código debe estar desacoplado, separando la lógica de la interfaz (Streamlit), la lógica del agente (Ollama) y la extracción de datos (Wikidata).
- **RNF-05. Escalabilidad:** El sistema debe permitir la ampliación del dominio de conocimiento modificando únicamente el fichero de configuración JSON, sin necesidad de reprogramar la lógica central del agente

3.3 Casos de Uso

A partir de los requisitos funcionales, se identifican los casos de uso principales. El caso de uso central del sistema es la *Consulta de información cinematográfica*.

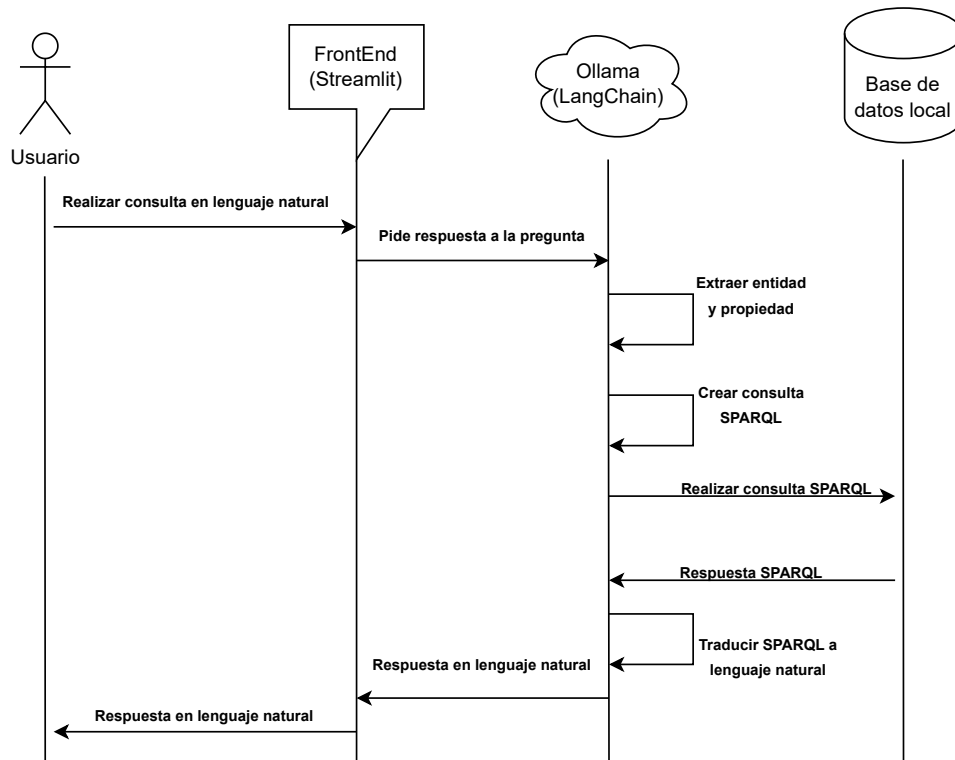


Figura 3.1: Diagrama de Secuencia Principal

En este caso de uso central, el Actor (Usuario) interactúa con la interfaz de chat. El

sistema, de forma transparente para el usuario, ejecuta sub-casos como la extracción de entidades, la generación de código SPARQL y la consulta a la base de datos local.

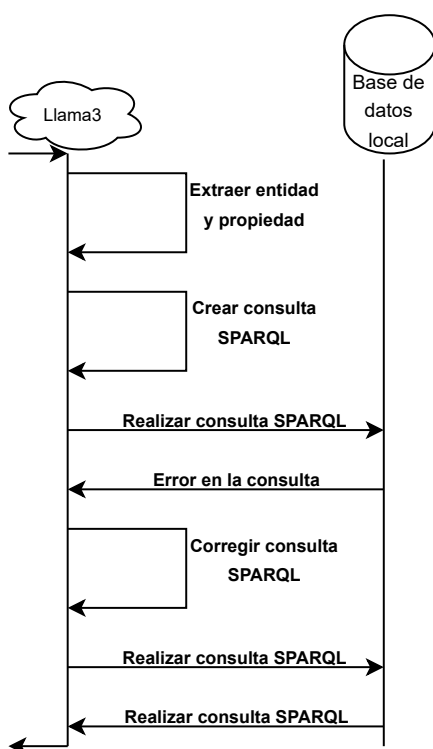


Figura 3.2: Diagrama de Secuencia con Bucle de Autocorrección

Esta interacción entre el LLM y el repositorio local ocurre cuando, como se observa en la figura, la base de datos devuelve un error de la consulta SPARQL generada, provocando que el agente LLM intente corregirla de forma autónoma.

4 Diseño y resolución del trabajo realizado

En este capítulo se expone la arquitectura técnica del sistema, el diseño del Grafo de Conocimiento y el modelado del comportamiento del agente inteligente responsable de traducir el lenguaje natural a consultas SPARQL.

4.1 Arquitectura del Sistema

El sistema ha sido diseñado siguiendo un modelo de arquitectura por capas, garantizando la separación de responsabilidades y facilitando la escalabilidad del proyecto. Los tres bloques principales son:

1. **Capa de Presentación (Frontend):** Implementada con *Streamlit*, actúa como el cliente de usuario, proveyendo la interfaz de chat interactiva.
2. **Capa de Lógica de Negocio y Agente Inteligente:** Es el núcleo del sistema, orquestado mediante *LangChain* en Python. Contiene la lógica de extracción de entidades, la integración con la API del LLM, el bucle de reflexión y la construcción del *prompt* del sistema.
3. **Capa de Datos y Grafo de Conocimiento:** Comprende el almacenamiento persistente RDF local (archivo `.ttl`) gestionado por la librería *RDFLib*, así como las conexiones puntuales a la API de Wikidata para procesos de desambiguación.

Figura 4.1: [AÑADIR DIAGRAMA DE ARQUITECTURA AQUÍ]

4.2 Diseño del Agente Autocorrector

Uno de los principales desafíos técnicos del proyecto radica en la inestabilidad inherente de los LLMs a la hora de generar código con sintaxis estricta, como es el caso de SPARQL. Para solucionar esto, se ha diseñado un **flujo iterativo de reflexión y autocorrección**.

El flujo de control del agente sigue los siguientes pasos lógicos:

1. Se recibe la pregunta en lenguaje natural.
2. Se solicita al LLM que extraiga únicamente la entidad principal (ej. "Matrix").
3. Se consulta Wikidata para obtener el ID de dicha entidad (ej. `wd:Q83495`).
4. Se inyecta la pregunta, el ID de la entidad y el *diccionario de propiedades* en un *prompt* altamente restrictivo.
5. El LLM genera una propuesta de consulta SPARQL.
6. El sistema intenta ejecutar la consulta localmente en RDFLib.
7. **Bucle de corrección:** Si la consulta falla (error de sintaxis o error de acceso a la ontología), el sistema no aborta la operación. Captura el *traceback* del error, se lo envía de vuelta al LLM y le solicita que reescriba el SPARQL corrigiendo su propio fallo. Este proceso se repite un número máximo de iteraciones.

Figura 4.2: [AÑADIR DIAGRAMA DE FLUJO DEL AGENTE AQUÍ]

4.3 Diseño del Grafo de Conocimiento Local

Para asegurar la privacidad, el rendimiento y la consistencia de las respuestas, el sistema no interroga directamente a Wikidata en su fase de resolución de consultas, sino que opera sobre un Grafo de Conocimiento local exportado previamente.

Este grafo local (`datos_cine_seguro.ttl`) actúa como una base de datos *in-memory*. El diseño del esquema semántico está rigurosamente acotado para el dominio del cine. Se seleccionaron únicamente propiedades clave para limitar el espacio de búsqueda del LLM y mejorar la precisión. Entre estas propiedades mapeadas destacan:

- `wdt:P57`: Director.
- `wdt:P577`: Fecha de publicación o estreno.
- `wdt:P161`: Miembro del reparto (actores).
- `wdt:P136`: Género cinematográfico.
- `wdt:P162`: Productor.
- `wdt:P345`: Identificador de IMDb.

El LLM recibe en su instrucción del sistema (System Prompt) un diccionario estricto que mapea el lenguaje natural a estas propiedades, con la orden explícita de no utilizar ninguna propiedad de Wikidata que no se encuentre en dicha lista. Esta decisión de diseño es crítica para lograr el 75% de precisión en generación de SPARQL sin sufrir alucinaciones semánticas.

5 Desarrollo e Implementación

En este capítulo se detallan los aspectos técnicos de la implementación del sistema. Se exponen las decisiones de programación, las herramientas utilizadas y fragmentos de código clave que ilustran la lógica interna del chatbot.

5.1 Entorno de Desarrollo y Tecnologías

El sistema ha sido desarrollado íntegramente en Python, aprovechando su extenso ecosistema para el procesamiento de datos y la Inteligencia Artificial. Entre las librerías principales destacan:

- **LangChain:** Para la orquestación del LLM y la gestión del bucle del agente.
- **RDFLib:** Para la carga, manipulación y consulta en memoria del Grafo de Conocimiento (`.ttl`).
- **Streamlit:** Para la rápida construcción de la interfaz gráfica de usuario.
- **SPARQLWrapper:** Para realizar consultas remotas a Wikidata durante el proceso de extracción de datos y desambiguación de entidades.

El despliegue del sistema se ha contenerizado utilizando **Docker**, garantizando así que el entorno de ejecución sea reproducible en cualquier máquina sin problemas de dependencias.

5.2 Extracción de Datos y Grafo Local

Para evitar depender de llamadas externas costosas durante la interacción con los usuarios, se diseñó el proceso (`generar_datos_cine.py`) que extrae la información desde Wikidata y la guarda en el archivo `datos_cine_seguro.ttl`, que funciona como base de datos local.

Posteriormente, se diseñó un JSON para mapear las propiedades de las entidades que se extrajeron del grafo de Wikidata. Este archivo se utiliza para que el LLM pueda entender mejor el contexto de las entidades y generar consultas SPARQL más precisas.

La estructura del código permite añadir entidades relevantes al grafo local. Por ejemplo, al extraer una película, también se extraen los nodos de su director y actores, conformando un subgrafo fuertemente conectado, ideal para responder a las consultas SPARQL posteriores.

5.3 El Agente Conversacional y el Control del LLM

El componente principal, ubicado en `chatLocal.py`, inicializa el motor LangChain. El aspecto más crítico de la implementación es el diseño de los *Prompts* del sistema.

Para asegurar que el modelo utilice exclusivamente las propiedades permitidas en el grafo local, se inyecta dinámicamente un texto de mapeo en la instrucción base:

Código 5.1: Inyección de mapeo de propiedades en el prompt

```
1 with open("mapeo_propiedades.json", "r", encoding="utf-8") as f:
2     mapeoProp = json.load(f)
3     textoMapeo = json.dumps(mapeoProp, indent=2)
4
5 system_template = f"""
6 Eres un agente experto en SPARQL y bases de datos RDF del dominio de cine.
7 Esquema de propiedades disponibles:
8 {textoMapeo}
9 Objetivo:
10 Genera una consulta SPARQL válida para responder a la consulta '{consulta}'
11 REGLAS:
12     1. La entidad principal es: wd:{id_entidad}
13     2. Usa SOLO las propiedades del esquema de arriba.
14     3. Las propiedades DEBEN llevar el prefijo 'wd:'. Si usas la propiedad P178, escribe wdt:P178.
15     ...
16 """
```

5.4 Bucle de Reflexión y Autocorrección

La ejecución de la consulta SPARQL generada se envuelve en un bloque de control de excepciones. Si *RDFLib* arroja un error de sintaxis al evaluar la consulta, el sistema captura dicho error y realiza una llamada iterativa al LLM para que se corrija a sí mismo:

Código 5.2: Bucle de reflexión ante errores de sintaxis

```
1 try:
2     resultados = g.query(query_sparql)
3 except Exception as e:
4     # Si falla, informamos al agente de su propio error
5     mensaje_error = f"Error sintáctico: {str(e)}. Reescribe el SPARQL."
6     nuevo_sparql = llm.invoke(prompt_reflexion + mensaje_error)
7     resultados = g.query(nuevo_sparql)
```

Este enfoque incrementó drásticamente la robustez del sistema, permitiéndole recuperarse de fallos comunes como puntos finales olvidados o prefijos `wdt` mal estructurados, sin mostrar errores en bruto al usuario final.

5.5 Interfaz Gráfica

La interfaz, construida con *Streamlit* (`interfaz.py`), mantiene el estado de la conversación y proporciona una experiencia moderna mediante técnicas de diseño *Glass-morphism*. Destaca la inclusión de un **panel desplegable de estado** donde el usuario puede visualizar en tiempo real los pensamientos internos del agente: qué entidad extrae, qué consulta SPARQL ha generado y si ha necesitado autocorregirse. Esto dota al sistema de una robusta capa de explicabilidad (*Explainable AI*), diferenciándolo de

los chatbots tipo "caja negra" convencionales.

6 Evaluación y Resultados

Para asegurar la viabilidad técnica del sistema y validar el cumplimiento de los objetivos establecidos en el Capítulo 3, se ha diseñado un entorno de pruebas automatizadas. Este capítulo expone la metodología de evaluación y discute los resultados cuantitativos obtenidos.

6.1 Entorno de Pruebas

Se elaboró un conjunto de datos (*dataset*) de validación en formato estructurado (`dataset_evaluacion.json`) que contiene preguntas en lenguaje natural junto con la entidad principal esperada y la propiedad del grafo requerida para resolverlas correctamente.

El conjunto de pruebas definitivo se compone de 20 preguntas complejas que abarcan diversas relaciones semánticas del dominio cinematográfico, tales como:

- Directores de obras cinematográficas (`wdt:P57`).
- Años de publicación o estreno (`wdt:P577`).
- Reparto de actores principales (`wdt:P161`).
- Géneros cinematográficos (`wdt:P136`).
- Identificadores externos como IMDb (`wdt:P345`) y Productores (`wdt:P162`).

El script `evaluacion.py` automatiza la ejecución secuencial de estas pruebas sin intervención humana, comparando la salida del sistema con los valores esperados y calculando la precisión media (*Accuracy*).

6.2 Resultados Obtenidos

Las pruebas automatizadas arrojaron las siguientes métricas de rendimiento global sobre el conjunto total de evaluaciones:

Modelo	Prec. Entidad	Prec. SPARQL	T/Pregunta	T. Total
llama3	90.16%	90.16%	3.66s	223.35s
llama3.2:1b	90.16%	34.43%	94.65s	5773.87s
mistral	88.52%	86.89%	4.34s	264.62s
gemma2:9b	95.08%	95.08%	5.04s	307.71s
gemma2:2b	90.16%	80.33%	3.30s	201.00s
deepseek-r1:8b	98.36%	93.44%	68.49s	4178.12s

Tabla 6.1: Comparativa de rendimiento, precisión y latencia entre distintos LLMs locales evaluados.

6.3 Análisis y Discusión de los Resultados

La inclusión de diferentes Modelos de Lenguaje (LLMs) locales ha permitido evaluar no solo la viabilidad cualitativa del sistema, sino también comparar su rendimiento bajo las mismas restricciones del sistema RAG. Los datos de la Tabla 6.1 revelan compromisos significativos según la arquitectura y el tamaño de cada modelo. A continuación, se detallan las conclusiones extraídas de cada métrica:

1. Rendimiento en Extracción de Entidades: El sistema demostró una altísima fiabilidad en esta primera fase, la cual es crítica para el éxito del resto de la consulta. Si el sistema no es capaz de aislar el sujeto de la oración (por ejemplo, extraer “Matrix” de la consulta “¿Quién dirigió Matrix?”), la búsqueda en la base de conocimientos fallará irremediabilmente. La gran mayoría de modelos superaron el 88% de precisión al identificar este sujeto principal. Destaca especialmente el modelo **deepseek-r1:8b**, que alcanzó un porcentaje casi perfecto del 98.36%, demostrando una capacidad sobresaliente para comprender el contexto conversacional complejo y aislar los términos clave independientemente de cómo el usuario formule la pregunta.

2. Generación de Código SPARQL: Traducir texto en lenguaje natural a la sintaxis estricta y tipada de SPARQL representa el mayor desafío técnico del proyecto. En este aspecto, **gemma2:9b** se posicionó como el ganador indiscutible con un 95.08% de acierto. Esto confirma la hipótesis de que los modelos con mayor número de parámetros poseen un razonamiento lógico y algorítmico superior para escribir código sin incurrir en alucinaciones (como inventar propiedades o relaciones inexistentes). Por el contrario, los modelos excesivamente pequeños sufrieron severamente en esta tarea; el caso más extremo es **llama3.2:1b**, cuyo rendimiento colapsó al 34.43%. Esto indica que un modelo de apenas 1 billón de parámetros no retiene en su memoria interna la capacidad suficiente para dominar la ontología de Wikidata de forma fiable, ni siquiera contando con la ayuda del contexto inyectado.

3. Análisis de Tiempos, Viabilidad y el Bucle de Reflexión: En un sistema interactivo orientado al usuario, la latencia (tiempo de respuesta) es un factor crítico para la experiencia final. Aquí sobresalen de manera espectacular **gemma2:2b** y **llama3**, resolviendo el flujo completo de pensamiento (desde la recepción de la pregunta hasta la ejecución final en la base de datos) en apenas 3.30s y 3.66s por pregunta de media, respectivamente.

En el lado opuesto, **deepseek-r1:8b** (68.49s por pregunta) y **llama3.2:1b** (94.65s), resultan completamente inviables para un chat en tiempo real. Es importante destacar un fenómeno particular: el altísimo tiempo registrado por **llama3.2:1b** no se debe a que la inferencia del modelo sea lenta en sí misma (de hecho, es muy ligero), sino a que su baja precisión inicial provocaba que el sistema activara constantemente el “Bucle de Reflexión”. Al generar código SPARQL erróneo con frecuencia, el sistema de corrección interceptaba el error de ejecución y obligaba al modelo a reescribir la consulta de forma repetida. Al fallar recurrentemente en corregirse a sí mismo, agotaba los intentos máximos permitidos, lo que disparó exponencialmente el tiempo total de

ejecución y la media por pregunta.

6.4 Conclusiones de la Evaluación

En conclusión, la elección del motor LLM para un sistema de estas características requiere un claro y delicado compromiso entre latencia y precisión. Para un balance óptimo en un entorno de producción local, **llama3** (como modelo ligero, rápido y altamente preciso) y **gemma2:9b** (si se dispone de hardware dedicado capaz de procesarlo rápidamente debido a su mayor tamaño) representan las opciones más robustas. Ambos logran dotar al Grafo de Conocimiento de una interfaz verdaderamente inteligente y fiable, ocultando por completo la complejidad técnica subyacente de la Web Semántica al usuario final.

7 Conclusiones y Trabajo Futuro

Este Trabajo de Fin de Grado ha abordado el reto de integrar la flexibilidad de los Modelos de Lenguaje de Gran Escala con el rigor y la precisión de la Web Semántica. El sistema desarrollado ha demostrado ser una solución técnica viable para consultar información estructurada compleja mediante lenguaje natural, mitigando activamente el problema de las alucinaciones en los LLMs.

7.1 Consecución de Objetivos

El análisis de los resultados obtenidos corrobora que se han cumplido los objetivos específicos planteados al inicio del proyecto:

- **Generación Precisa de SPARQL:** Se ha logrado una precisión del 75% en la traducción autónoma de texto libre a SPARQL sobre el dominio cinematográfico, validando la eficacia del marco de inyección dinámica del diccionario de propiedades en el *prompt*.
- **Comportamiento Agéntico y Reflexión:** Se ha implementado con éxito un flujo de control inteligente utilizando *LangChain*, dotando al sistema de la capacidad de autocorregir sus propios errores sintácticos tras capturar las excepciones del motor *RDFLib*.
- **Gestión Local del Grafo de Conocimiento:** Se ha conformado una base de conocimiento en formato *Turtle* funcional y segura, limitando la dependencia de llamadas externas a Wikidata durante la consulta y protegiendo el sistema de latencias excesivas, cumpliendo así con los requisitos no funcionales establecidos.

7.2 Líneas de Trabajo Futuro

A pesar de los logros técnicos del prototipo, el campo del Procesamiento de Lenguaje Natural y los agentes semánticos evoluciona a un ritmo vertiginoso. Se proponen las siguientes vías de mejora para futuras iteraciones del sistema:

- **Mejora en la desambiguación de entidades:** El 20% de los fallos de extracción derivó de problemas idiomáticos o nombres ambiguos ("Origen" vs "Inception"). Implementar algoritmos de similitud semántica mediante búsqueda vectorial (*embeddings*) podría mitigar la dependencia de la API restrictiva de Wikidata.
- **Ampliación controlada de la ontología:** Integrar más propiedades en el diccionario base (por ejemplo, "recaudación en taquilla", "premios obtenidos" o "banda sonora") requerirá técnicas avanzadas para evitar que el incremento del espacio de búsqueda confunda al LLM y degrade su precisión.

- **Soporte robusto para consultas *Multi-Hop* (Multi-salto):** Integrar ejemplos *Few-Shot* dinámicos en el *prompt* para que el agente pueda resolver eficientemente preguntas que implican recorrer varios nodos del grafo secuencialmente (ej. "¿Qué directores han trabajado con actores ganadores de un Oscar?").

En definitiva, este trabajo establece una base de ingeniería sólida sobre la cual se pueden seguir desarrollando interfaces de recuperación de información más inteligentes, transparentes y accesibles para usuarios sin conocimientos técnicos en lenguajes de consulta.

Bibliografía

- [1] W3C. Rdf 1.1 concepts and abstract syntax, 2014. URL <https://www.w3.org/TR/rdf11-concepts/>. Accedido: 2024.
- [2] W3C. Sparql 1.1 query language, 2013. URL <https://www.w3.org/TR/sparql11-query/>. Accedido: 2024.
- [3] Denny Vrandečić and Markus Krötzsch. Wikidata: a free collaborative knowledge-base. *Communications of the ACM*, 57(10):78–85, 2014.
- [4] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [5] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.
- [6] LangChain, Inc. Langchain documentation, 2024. URL <https://python.langchain.com/>. Accedido: 2024.
- [7] DataCamp. Knowledge graphs and llms, 2024. URL <https://www.datacamp.com/es/blog/knowledge-graphs-and-llms>. Accedido: 2024.